

Produtividade

Introdução

A produtividade pode ser entendida, de maneira geral, como a capacidade de produzir determinado resultado utilizando os recursos disponíveis de forma eficiente. Na economia, ela está relacionada à relação entre aquilo que é produzido e os recursos utilizados para sua produção, como mão de obra, materiais, energia, capital e tempo. Dessa forma, ser produtivo não significa necessariamente trabalhar mais, mas utilizar os recursos disponíveis de maneira eficiente para alcançar determinado resultado.

O conceito de produtividade possui uma longa história. Embora atividades produtivas existam desde a pré-história, o termo produtividade surgiu em 1766, associado ao economista francês François Quesnay. Posteriormente, o conceito ganhou força durante a Revolução Industrial, principalmente na Inglaterra, quando as fábricas passaram a buscar maneiras de aumentar a produção. No início do século XX, Henry Ford contribuiu para esse processo com a implantação da linha de montagem e da produção em massa. A partir da década de 1970, o modelo desenvolvido pela Toyota passou a ganhar destaque por valorizar o trabalho em equipe, a qualificação dos trabalhadores e a redução de desperdícios.

Com o avanço da tecnologia, a produtividade passou por novas transformações. Computadores, internet e dispositivos móveis possibilitaram que diversas tarefas fossem realizadas de maneira mais rápida. Ao mesmo tempo, essas tecnologias aumentaram a quantidade de demandas e a expectativa de que os trabalhadores estejam constantemente disponíveis. Dessa forma, a busca por produtividade também pode gerar pressão, principalmente quando o desempenho passa a ser associado apenas à quantidade de tarefas realizadas.

Na área de desenvolvimento de software, essa discussão torna-se ainda mais relevante com o crescimento da Inteligência Artificial (IA). Ferramentas capazes de gerar código, explicar problemas, criar testes e auxiliar na manutenção de sistemas passaram a fazer parte da rotina de muitos desenvolvedores. Entretanto, a utilização dessas ferramentas levanta uma questão importante: se a Inteligência Artificial permite realizar determinadas tarefas mais rapidamente, isso significa necessariamente que o programador se tornou mais produtivo?

Produtividade

Produtividade no desenvolvimento de software

Ser produtivo no desenvolvimento de software não significa simplesmente escrever uma quantidade maior de código em menos tempo. O desenvolvimento de sistemas envolve diversas atividades, como planejamento, programação, testes, correção de erros, revisão de código, documentação, manutenção e comunicação entre os membros da equipe.

Por esse motivo, medir produtividade nessa área é uma tarefa complexa. Uma métrica isolada pode apresentar apenas uma parte do processo. Por exemplo, a quantidade de linhas de código produzidas pode aumentar, mas isso não significa necessariamente que o software tenha maior qualidade. Um código maior também pode conter mais erros, exigir mais manutenção ou gerar retrabalho.

A DORA, programa de pesquisa voltado ao estudo do desempenho de desenvolvimento e operações de software, trabalha com uma visão mais ampla das capacidades que influenciam a entrega de software. A organização também destaca que a adoção de Inteligência Artificial pode apresentar uma chamada "productivity dip", ou queda inicial de produtividade, durante a implementação e adaptação das novas ferramentas. ([Dora](#))

Assim, velocidade e produtividade não devem ser tratadas como sinônimos. A velocidade pode ser um indicador importante, mas produtividade envolve também o resultado alcançado, os recursos utilizados e a qualidade desse resultado.

Produtividade

Inteligência Artificial e produtividade na programação

A Inteligência Artificial pode auxiliar os programadores em diferentes etapas do desenvolvimento de software. Ferramentas de IA podem gerar trechos de código, sugerir soluções, explicar erros, produzir testes e auxiliar em tarefas repetitivas. A utilização dessas ferramentas pode reduzir o tempo necessário para determinadas atividades e permitir que o desenvolvedor concentre mais atenção em problemas de maior complexidade.

Um exemplo estudado empiricamente é o GitHub Copilot. Peng et al. (2023) realizaram um experimento controlado no qual desenvolvedores deveriam implementar um servidor HTTP em JavaScript. O grupo que teve acesso ao Copilot concluiu a tarefa 55,8% mais rapidamente que o grupo de controle. O resultado demonstra um ganho significativo de velocidade em uma tarefa específica.

Esse resultado, entretanto, não deve ser interpretado como prova de que a IA sempre aumenta a produtividade dos programadores. O próprio estudo apresenta efeitos heterogêneos, mostrando que os resultados podem variar entre diferentes pessoas e situações.

Essa questão torna-se ainda mais evidente quando outros experimentos são considerados. Becker et al. (2025) realizaram um experimento controlado randomizado com 16 desenvolvedores experientes, que realizaram 246 tarefas em projetos maduros de código aberto nos quais possuíam, em média, cinco anos de experiência prévia. Os pesquisadores permitiram o uso de ferramentas de IA em parte das tarefas e proibiram seu uso em outras.

Antes do experimento, os desenvolvedores esperavam que a IA reduzisse o tempo de conclusão das tarefas em aproximadamente 24%. Após realizarem as tarefas, estimaram uma redução de 20%. Entretanto, o resultado observado foi diferente: nas condições do experimento, permitir o uso das ferramentas de IA aumentou o tempo necessário para concluir as tarefas em 19%.

Esse resultado é importante porque demonstra que a percepção de produtividade e a produtividade medida podem ser diferentes. Uma ferramenta pode parecer útil e proporcionar respostas ou códigos rapidamente, mas o tempo necessário para compreender, revisar, testar e integrar essas respostas ao projeto também precisa ser considerado.

Produtividade

Por que é difícil medir a produtividade de um programador?

A dificuldade de medir produtividade em programação está relacionada à própria natureza do trabalho. Diferentemente de uma atividade em que é possível contar facilmente a quantidade produzida, o desenvolvimento de software envolve tarefas intelectuais e resultados que podem variar muito em complexidade.

Um programador pode passar várias horas analisando um problema antes de escrever uma única linha de código. Mesmo assim, essa análise pode ser fundamental para evitar erros futuros. Da mesma forma, uma alteração pequena em um sistema pode exigir conhecimentos técnicos e testes complexos.

Por isso, métricas baseadas apenas em quantidade ou velocidade podem gerar interpretações equivocadas. Uma pessoa pode escrever mais código, mas produzir uma solução menos eficiente ou com mais problemas. Da mesma maneira, outra pessoa pode produzir menos código, mas resolver um problema mais complexo de forma adequada.

A DORA busca analisar o desempenho de software considerando um conjunto mais amplo de fatores, reforçando a necessidade de uma visão sistêmica sobre o processo de desenvolvimento.

No caso da IA, essa dificuldade aumenta porque parte do trabalho passa a ser realizada por ferramentas automatizadas. Assim, medir apenas a quantidade de código escrito diretamente pelo programador deixa de representar todo o processo.

Produtividade

A experiência da Spotify com agentes de programação

Um exemplo prático da utilização de IA em larga escala pode ser encontrado na experiência da Spotify com agentes de programação. A empresa desenvolveu uma infraestrutura chamada Fleet Management para automatizar alterações em seus diversos repositórios. Antes da utilização de agentes de IA, o sistema já era capaz de automatizar tarefas repetitivas, como atualização de dependências, arquivos de configuração e algumas refatorações simples. Desde meados de 2024, aproximadamente metade dos pull requests da empresa era automatizada por esse sistema.

Posteriormente, a Spotify passou a utilizar agentes de IA para realizar alterações mais complexas. Em seu relato de 2025, a empresa informou que seus agentes haviam gerado mais de 1.500 pull requests que foram incorporados ao código de produção. Entre os exemplos estão modernização de linguagens, atualizações com mudanças incompatíveis, migrações de componentes de interface e alterações de configurações. A empresa também relatou uma economia de tempo de aproximadamente 60% a 90% em determinadas migrações quando comparadas à implementação manual.

Entretanto, a própria Spotify destaca que esses ganhos não eliminam os desafios. Os agentes podem demorar para produzir resultados, apresentar comportamentos imprevisíveis e exigir novos mecanismos de validação e controle de qualidade. A empresa também aponta questões relacionadas à segurança, aos mecanismos de proteção e ao custo computacional necessário para executar modelos de linguagem em escala.

Esse exemplo mostra novamente que produtividade não deve ser analisada apenas pela velocidade. A automação pode reduzir o tempo de determinadas atividades, mas também cria novas necessidades relacionadas à revisão, validação e controle.

Produtividade

A percepção de produtividade

Outro aspecto importante é a diferença entre a sensação de estar produzindo mais e o aumento efetivo da produtividade.

A percepção do programador é relevante, mas não pode ser utilizada isoladamente para determinar o desempenho. O estudo de Becker et al. (2025) apresenta um exemplo significativo dessa diferença. Os desenvolvedores participantes esperavam que o uso de IA diminuísse o tempo de realização das tarefas e, depois da experiência, continuaram estimando que haviam obtido uma redução de tempo. No entanto, a medição realizada pelos pesquisadores mostrou aumento de 19% no tempo de conclusão nas condições estudadas.

Isso demonstra que uma ferramenta pode transmitir uma sensação de eficiência sem necessariamente diminuir o tempo total necessário para entregar um resultado final.

Na programação, isso pode acontecer porque a IA fornece uma resposta imediatamente, mas o desenvolvedor ainda precisa verificar se ela está correta, compreender o código, realizar testes e adaptar a solução ao sistema existente. Portanto, o ganho percebido em uma etapa não representa necessariamente um ganho no processo completo.

Produtividade

Inteligência Artificial, Machine Learning e produtividade

A Inteligência Artificial é um campo amplo que inclui diferentes técnicas e aplicações. Entre elas está o Machine Learning (ML), ou aprendizado de máquina, que permite que sistemas identifiquem padrões a partir de dados e utilizem esses padrões para realizar determinadas tarefas.

No desenvolvimento de software, sistemas baseados em IA e ML podem ser utilizados para auxiliar na análise de código, identificação de padrões, previsão de problemas, geração de sugestões e automação de tarefas. Com a evolução dos modelos generativos, também se tornou possível produzir código e outros artefatos de software a partir de instruções em linguagem natural.

Esse processo modifica a relação entre o programador e a ferramenta. Em vez de escrever manualmente todas as instruções necessárias, o desenvolvedor pode fornecer uma descrição do que deseja realizar e utilizar a IA como parte do processo de desenvolvimento.

Entretanto, isso não significa que o conhecimento técnico deixe de ser necessário. O desenvolvedor ainda precisa avaliar a solução, compreender suas consequências, testar o funcionamento e verificar se o código atende aos requisitos do projeto. A experiência da Spotify demonstra essa necessidade ao incorporar mecanismos de linting, avaliação de diferenças, registros e outras formas de controle ao fluxo dos agentes de programação.

Produtividade

O Paradoxo de Jevons aplicado à Inteligência Artificial

O Paradoxo de Jevons ajuda a ampliar a discussão sobre eficiência e produtividade. O conceito foi originalmente associado às melhorias na eficiência das máquinas a vapor e à possibilidade de que ganhos de eficiência levassem, paradoxalmente, a um aumento no consumo total do recurso.

Na área de Inteligência Artificial, essa ideia pode ser utilizada para discutir os chamados efeitos de rebote. Melhorias na eficiência podem diminuir o custo ou o tempo necessário para realizar determinada atividade, mas isso pode estimular um aumento no uso da tecnologia.

Aplicado ao desenvolvimento de software, o raciocínio pode ser apresentado da seguinte maneira: se a IA permite que uma tarefa de programação seja realizada mais rapidamente, isso não significa necessariamente que os programadores passarão a trabalhar menos. A redução do custo de produção pode permitir que empresas realizem mais projetos, desenvolvam mais funcionalidades, façam mais alterações e mantenham uma quantidade maior de software.

Essa possibilidade é particularmente relevante quando observamos a experiência da Spotify. A empresa não utiliza os agentes apenas para acelerar uma tarefa individual, mas para aplicar mudanças em grande escala em seus repositórios. Os agentes já produziram mais de 1.500 pull requests incorporados à produção, e a empresa afirma que o retorno das automações aumenta à medida que elas são aplicadas a mais bases de código.

Pesquisas recentes também discutem explicitamente a relação entre IA e o Paradoxo de Jevons. Luccioni, Strubell e Crawford (2025), por exemplo, argumentam que ganhos de eficiência em IA podem gerar efeitos de rebote e que a análise de seus impactos precisa considerar efeitos indiretos, além do consumo direto de recursos.

Dessa maneira, o Paradoxo de Jevons não significa que todo ganho de eficiência necessariamente produzirá maior consumo. Ele serve como uma possibilidade a ser considerada: quando uma tecnologia se torna mais eficiente e acessível, o aumento da demanda pode compensar parte ou até superar a economia obtida por unidade de uso.

Produtividade

Considerações finais

A análise da produtividade no desenvolvimento de software demonstra que velocidade, quantidade de código e sensação de eficiência não são suficientes para determinar se um programador ou uma equipe realmente se tornou mais produtiva. A produtividade precisa considerar o resultado produzido, os recursos utilizados, a qualidade, o retrabalho e as características da tarefa.

Os estudos analisados apresentam resultados diferentes sobre o impacto da Inteligência Artificial. O experimento com GitHub Copilot encontrou uma redução de 55,8% no tempo para a realização de uma tarefa específica, enquanto o estudo de Becker et al. (2025), realizado com desenvolvedores experientes em projetos de código aberto, encontrou aumento de 19% no tempo de conclusão nas condições estudadas.

Esses resultados não precisam ser vistos como contraditórios. Eles demonstram que o impacto da IA depende do contexto, do tipo de tarefa, da experiência do desenvolvedor, das ferramentas utilizadas e da maneira como a produtividade é medida.

A experiência da Spotify reforça essa conclusão ao apresentar ganhos expressivos de tempo em determinadas migrações, mas também destacar desafios relacionados à imprevisibilidade, validação, segurança e custo computacional dos agentes.

Portanto, a Inteligência Artificial pode ser uma importante ferramenta de apoio à produtividade na programação, mas seu impacto não deve ser avaliado somente pela velocidade com que produz código. É necessário analisar o processo completo e verificar se os ganhos de eficiência realmente se transformam em melhores resultados.

Além disso, o Paradoxo de Jevons mostra que aumentar a eficiência não significa necessariamente diminuir o uso total de recursos ou a quantidade de trabalho. No desenvolvimento de software, uma IA mais eficiente pode permitir que uma equipe produza e mantenha uma quantidade muito maior de sistemas e funcionalidades. Dessa forma, compreender a produtividade na era da Inteligência Artificial exige analisar não apenas quanto tempo uma tarefa economiza, mas também o que passa a ser feito com essa capacidade adicional.

Referências principais

- AKITA, Fabio. Do zero à pós-produção em 1 semana: como usar IA em projetos de verdade — bastidores do The-M Akita Chronicles. 2026. Disponível em: Akita on Rails
- BECKER, Joel; RUSH, Nate; BARNES, Elizabeth; REIN, David. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. 2025. Disponível em: arXiv
- DORA. DORA — DevOps Research and Assessment. Google Cloud. Disponível em: DORA
- LUCCIONI, Alexandra Sasha; STRUBELL, Emma; CRAWFORD, Kate. From Efficiency Gains to Rebound Effects: The Problem of Jevons' Paradox in AI's Polarized Environmental Debate. 2025. Disponível em: arXiv
- PENG, Sida; KALLIAMVAKOU, Eirini; CIHON, Peter; DEMIRER, Mert. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. 2023. Disponível em: arXiv
- PUCRS ONLINE. Produtividade: o que é e como aplicar na sua carreira. Disponível em: PUCRS Online
- REVISTA GALILEU. Como o conceito de produtividade mudou desde a Revolução Industrial. 2019. Disponível em: Revista Galileu
- SPOTIFY ENGINEERING. Spotify's Background Coding Agent — Part 1. 2025. Disponível em: Spotify Engineering
- SISTEMA EMPRAD. Gestão da produtividade em operações de serviços com aprendizagem de máquina e sistemas de informação. Anais do EMPRAD. Disponível em: EMPRAD
- TERRA. Produtividade excessiva: a nova demanda da vida moderna. 2025. Disponível em: Terra